

Esercizi C#

Programmazione Orientata agli Oggetti

Ereditarietà · Gerarchia di classi · Polimorfismo
Uso di liste · Classi astratte · Interfacce

10 esercizi · difficoltà progressiva

Esercizio 01

Animali che parlano

Ereditarietà — Facile

Crea una gerarchia di classi per rappresentare animali. La classe base `Animale` ha nome e un metodo `FaiVerso()`. Le classi derivate `Cane`, `Gatto` e `Mucca` sovrascrivono il metodo con il verso corretto.

COSA IMPLEMENTARE

1. Crea la classe base `Animale` con proprietà `Nome` e metodo virtuale `FaiVerso()`.
2. Deriva `Cane`, `Gatto` e `Mucca` da `Animale`, sovrascrivendo `FaiVerso()` con il verso appropriato.
3. In `Main`, crea un oggetto per ogni tipo e chiama `FaiVerso()` su ciascuno.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi la proprietà `NumeroZampe` (int) nella classe base.
- Stampa le informazioni di ogni animale sovrascrivendo `ToString()`.

Esercizio 02

Gerarchia di veicoli

Ereditarietà · Gerarchia classi — Facile

Modella una gerarchia di veicoli a due livelli. `Veicolo` è la classe radice, da cui derivano `VeicoloTerrestre` e `VeicoloNavale`. Da questi derivano classi specifiche come `Auto`, `Camion`, `Barca`.

COSA IMPLEMENTARE

1. Crea `Veicolo` con `Marca`, `Anno` e metodo `Descrizione()`.
2. Crea `VeicoloTerrestre` (aggiungi `NumeroRuote`) e `VeicoloNavale` (aggiungi `Stazza`) come classi intermedie.
3. Crea almeno due classi concrete per ciascun ramo (es. `Auto` e `Camion`) e istanziale in `Main`.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi `VeicoloAereo` con la proprietà `AltitudineMax`.
- Crea un metodo statico che, data una lista di `Veicolo`, stampa solo quelli prodotti dopo il 2010.

Esercizio 03

Lista di studenti

Uso di liste — Facile

Gestisci un registro scolastico usando List. Ogni Studente ha nome, cognome e una lista di voti. Implementa operazioni di aggiunta, rimozione e calcolo della media.

COSA IMPLEMENTARE

1. Crea la classe Studente con Nome, Cognome e List Voti.
2. In Main crea una List, aggiungi almeno 4 studenti ciascuno con più voti.
3. Stampa la media voti di ogni studente.
4. Rimuovi dalla lista gli studenti con media inferiore a 6 e stampa la lista aggiornata.

PUNTI BONUS — per chi vuole andare oltre

- Ordina la lista per media decrescente e stampala.
- Trova e stampa lo studente con la media più alta.

Esercizio 04

Forme geometriche astratte

Classi astratte · Polimorfismo — Medio

Crea una classe astratta Forma con metodi astratti Area() e Perimetro(). Implementa Cerchio, Rettangolo e Triangolo come classi concrete che forniscono il calcolo specifico.

COSA IMPLEMENTARE

1. Dichiarare Forma come classe astratta con metodi abstract double Area() e abstract double Perimetro().
2. Implementa le tre forme concrete con i costruttori e le proprietà appropriate.
3. Crea un array di Forma con istanze miste e stampa area e perimetro di ognuna.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi un metodo virtuale Descrivi() nella classe astratta che utilizza Area() e Perimetro().
- Usa LINQ per trovare la forma con l'area massima all'interno della lista.

Esercizio 05

Interfaccia stampabile

Interfacce — Medio

Definisci un'interfaccia IStampabile con un metodo Stampa(). Implementala su classi diverse e non correlate — Fattura, Email, Report — e dimostra il polimorfismo tramite interfaccia.

COSA IMPLEMENTARE

1. Dichiarare l'interfaccia IStampabile con il metodo void Stampa().
2. Creare Fattura, Email e Report (classi senza relazione di ereditarietà tra loro) che implementano IStampabile.
3. In Main creare una List con oggetti misti e chiamare Stampa() su ciascuno.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungere una seconda interfaccia IEsportabile con string Esporta() e farla implementare solo da Fattura e Report.
- Usare un'interfaccia INotificabile con Notifica(string msg) e implementarla su Email.

Esercizio 06

Negozi di prodotti

Ereditarietà · Uso di liste · Gerarchia classi — Medio

Costruisci un sistema di gestione prodotti. Prodotto è la classe base. ProdottoAlimentare e ProdottoElettronico la estendono con proprietà specifiche. La classe Magazzino gestisce una lista di prodotti.

COSA IMPLEMENTARE

1. Creare la gerarchia: Prodotto come base, ProdottoAlimentare con DataScadenza e ProdottoElettronico con GaranziaAnni.
2. Creare la classe Magazzino con List e metodi Aggiungi, Rimuovi(string nome) e Cerca(string nome).
3. Stampare tutti i prodotti alimentari scaduti e gli elettronici con garanzia scaduta.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungere il metodo ValoreTotale() al Magazzino che somma i prezzi di tutti i prodotti.
- Implementare IComparable per ordinare i prodotti per prezzo.

Dipendenti e stipendi

[Classi astratte](#) · [Polimorfismo](#) · [Gerarchia classi](#) — Difficile

Modella una struttura aziendale. Dipendente è una classe astratta con Nome, Matricola e il metodo astratto CalcolaStipendio(). Impiegato ha uno stipendio fisso, Venditore ha base più commissioni, Manager ha fisso più bonus.

COSA IMPLEMENTARE

1. Dichiarare Dipendente come classe astratta con abstract decimal CalcolaStipendio().
2. Implementa Impiegato, Venditore (con VenditeMensili e percentuale commissione) e Manager (con Bonus).
3. Crea una lista di dipendenti misti, calcola e stampa lo stipendio di ognuno.
4. Calcola e stampa la somma totale degli stipendi dell'azienda.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi override string ToString() su ogni sottoclasse per una stampa leggibile.
- Usa LINQ per trovare il dipendente con lo stipendio più alto e calcolare la media aziendale.

Dispositivi smart — interfacce multiple

[Interfacce](#) · [Uso di liste](#) — Difficile

Modella dispositivi smart domestici usando interfacce multiple. IAccendibile ha Accendi() e Spegni(), IConnettibile ha Connetti(string rete), IControllabile ha ImpostaLivello(int livello). Ogni dispositivo implementa le interfacce appropriate.

COSA IMPLEMENTARE

1. Definisci le tre interfacce: IAccendibile, IConnettibile, IControllabile.
2. Crea Lampadina (IAccendibile, IControllabile), Termostato (IAccendibile, IControllabile) e Speaker (IAccendibile, IConnettibile, IControllabile).
3. Crea una List e spegni tutti i dispositivi. Crea una List e connetti tutti alla stessa rete.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi le proprietà IsAcceso (bool) e LivelloCorrente (int) e stampale per ogni dispositivo.
- Scrivi un metodo generico EseguiSuTutti(List lista, Action azione) e usalo per le operazioni di massa.

Gioco di ruolo — sistema di personaggi

[Classi astratte](#) · [Interfacce](#) · [Ereditarietà](#) · [Uso di liste](#) — Difficile

Costruisci il sistema di personaggi per un gioco di ruolo. Personaggio è astratto con Vita, Attacco, Difesa e metodo astratto AttaccoSpeciale(). L'interfaccia ICurabile definisce Cura(int punti). Guerriero, Mago e Paladino estendono Personaggio; il Paladino implementa anche ICurabile.

COSA IMPLEMENTARE

1. Crea la classe astratta Personaggio e l'interfaccia ICurabile.
2. Implementa Guerriero, Mago e Paladino con attacchi speciali diversi tra loro.
3. Il Paladino implementa ICurabile e può curare altri personaggi.
4. Simula un combattimento: crea un party di 3 personaggi che si sfidano in round, riducendo la vita a ogni attacco.

PUNTI BONUS — per chi vuole andare oltre

- Aggiungi un'interfaccia IVelenabile con ApplicaVeleno() e implementala sul Mago.
- Gestisci la morte: rimuovi dalla lista i personaggi con vita ≤ 0 e dichiara il vincitore.
- Aggiungi un inventario List con oggetti che modificano le statistiche del personaggio.

Mini e-commerce — catalogo e carrello

[Classi astratte](#) · [Interfacce](#) · [Ereditarietà](#) · [Gerarchia classi](#) · [Uso di liste](#) — Avanzato

Progetta un sistema e-commerce che integra tutti gli argomenti del corso. Gerarchia prodotti con prezzi polimorfici, interfacce per pagamento e spedizione, carrello con gestione delle quantità.

COSA IMPLEMENTARE

1. Gerarchia: Prodotto astratto con abstract decimal PrezzoFinale(), derivato in ProdottoFisico (peso, spese spedizione) e ProdottoDigitale (download).
2. Interfacce: IPagabile con Paga(decimal importo) implementata da CartaCredito e PayPal; ISpedibile con CalcolaSpedizione() su ProdottoFisico.
3. Classe Carrello con List<(Prodotto, int quantita)>, metodi Aggiungi, Rimuovi e TotaleCarrello().
4. In Main: crea il catalogo, aggiungi prodotti al carrello, mostra il totale e simula il pagamento.

PUNTI BONUS — per chi vuole andare oltre

- Applica il pattern Strategy per gli sconti: interfaccia IScontoStrategy con CalcolaSconto(decimal prezzo), implementa ScontoPercentuale e ScontoFisso.
 - Aggiungi la gestione del magazzino: ogni ProdottoFisico ha Giacenza; il carrello lancia un'eccezione se si supera la disponibilità.
 - Implementa ICloneable su Prodotto per duplicare articoli nel catalogo.
-